

Comparative Analysis of Performance and Energy Consumption of Open-Source MQTT Brokers

Shrijan Subedi

School of Built Environment, Engineering, and Computing
Leeds Beckett University, Leeds, United Kingdom

Abstract—With the widespread growth of Internet of Things (IoT), there is a growing concern towards making IoT infrastructures more and more efficient. Message Queuing Telemetry Transport (MQTT) brokers play central role in these infrastructures. Thus, in this study, the performance and energy efficiency of four open-source MQTT brokers was evaluated under a constrained hardware setup. Each broker was subjected to a stress test using a benchmarking tool. Performance and power usage data were collected to perform statistical analysis and provide recommendations. The analysis revealed that some brokers consume significantly more energy than others and correspondingly produce more Carbon Dioxide (CO₂) emissions. NanoMQ was found to be the most energy-efficient broker while HiveMQ CE consumed the most energy. The performance of all brokers was similar when publishing messages but there were some variations while subscribing to messages.

Index Terms—Internet of Things, MQTT broker, Energy Efficiency, CO₂ Emissions

I. INTRODUCTION

A. Background of Context

Internet of Things (IoT) devices can play a vital role in reducing carbon footprint and increasing the efficiency of various infrastructures. But the carbon footprint of the devices itself becomes significant considering the increasing number of smart connected devices [1]. It is estimated that around half of the internet connected devices will be IoT devices, resulting in an increase of carbon emissions worldwide [2]. Further, with the move towards edge computing and edge AI, a lot of computation must be performed on IoT hardware which traditionally was performed on the cloud [3], [4].

Various communication protocols exist to support communication in an IoT infrastructure, but MQTT remains widely adopted due to its efficiency [3]. It uses a publish/subscribe communication model over Transmission Control Protocol / User Datagram Protocol (TCP/UDP). MQTT brokers are intermediaries that facilitate communication between sensors and various network nodes [5]. Due to brokers being a central component in an IoT infrastructure, it is only logical to evaluate the performance and efficiency of the great number of available solutions, especially in the context of edge computing and resource constrained devices such as Raspberry Pi [5].

B. Aim and Research Objectives

The major objective of this research is to compare the performance and energy-efficiency of four open-source brokers and provide recommendations based on the findings of results

of test performed. The following research objectives have been formulated based on this:

- R01: Perform literature review.
 - R1.1: Carry out literature review on MQTT protocol.
 - R1.2: Carry out literature review on various MQTT brokers.
 - R1.3: Carry out literature review about the evaluation of MQTT brokers.
 - R1.4: Carry out literature review on MQTT Benchmarking tools.
- R02: Conduct experiments and collect data.
 - R2.1: Conduct the tests.
 - R2.2: Collect data regarding performance of the brokers.
 - R2.3: Collect data regarding energy consumption of the brokers.
- R03: Perform analysis on the data collected in R02.
- R04: Provide conclusions and recommendation for future work based on analysis.

C. Rationale

There has been a lot of research into comparing performance of MQTT brokers implementations and cloud solutions as is further discussed in the literature review in the coming section. However, there is still not much research that compares the energy efficiency of those implementations. This research aims to introduce a factor of sustainability and energy efficiency into the comparison. The conclusions deduced from this research can provide valuable insights into selection of MQTT brokers and allow various infrastructures to develop a more sustainable IoT implementation.

D. Contribution of the Research

This study has a focus on energy efficiency of various MQTT broker implementations within a constrained hardware such as a Raspberry-pi. The experiment performs a stress test on the broker using the throughput scenario taken from the article posted by the EMQX team [6]. Such a scenario can reflect a real life IoT configuration and provide valuable insights into the sustainability of such infrastructures.

II. LITERATURE REVIEW

A. MQTT Protocol

There are several application-level protocols available for IoT communication. MQTT is popular protocol used in IoT

applications due to being lightweight and energy-efficient compared to other protocols [7].

MQTT uses a publish subscribe model which is more suitable for machine-to-machine communication. This model consists of a publisher which generates the messages to be published. For example, a temperature sensor publishing the current temperature every second. The message is published to a specific topic, used to route the message. Subscribers are clients that subscribe to specific topics and receive the messages published in that topic [7].

The central component to this publish and subscribe architecture are MQTT brokers. MQTT brokers forward the published messages to subscribers of the topic. The publishers and subscribers communicate through this central broker [8]. MQTT brokers can either be hosted on the cloud or on edge devices such as a Raspberry pi [9].

There are various parameters that influence the communication in MQTT based infrastructure. Quality of Service (QoS) enforces how the message is published or received by the subscribers [7]. Different QoS levels have different performance results [10]. Security also influences the performance of MQTT communication, as communicating without security mechanisms such as TLS improves the performance of the broker [11].

B. Selection of MQTT Brokers

There is a wide selection of MQTT brokers available for IoT. In this research we focus on four open-source MQTT brokers NanoMQ, EMQX, HiveMQ and Mochi-MQTT.

Mochi-MQTT is an embeddable MQTT broker written in Go programming language. The broker is lightweight and supports version 5 of MQTT [12]. NanoMQ is another lightweight broker that is maintained by EMQ Technologies. It is compliant with MQTT 5 and is suitable for edge devices. It is implemented in the C programming language [13]. EMQX is high-performance broker maintained by EMQ Technologies implemented in Erlang programming language. But it is a scalable and feature broker that supports up to 100 million MQTT connections, as opposed to NanoMQ which is a more lightweight broker [14]. HiveMQ offers an open-source edition of their broker, HiveMQ community edition. It is a Java-based scalable MQTT broker and supports MQTT protocol version 5 (HiveMQ, n.d.).

A comparison of the brokers has been provided in table I.

TABLE I: Comparison of various brokers

Brokers	MQTT Version?	Language	QoS Support
Mochi-MQTT	3.1.1, 5.0	Go	0,1,2
EMQX	3.X, 5.0	Erlang	0,1,2
HiveMQ	3.1.1, 5.0	Java	0,1,2
NanoMQ	3.X, 5.0	C	0,1,2

The selection diversifies the programming language of each broker as well as the scale in which the broker is intended to be deployed. NanoMQ and Mochi-MQTT are both lightweight and suited for constrained devices whereas HiveMQ and EMQX are scalable brokers that support high traffic.

C. Evaluating MQTT Brokers

There has been numerous research evaluating the performance of MQTT brokers in various environments.

[15] tested various MQTT brokers - EMQX, Mosquitto, VerneMQ and HiveMQ and concluded that the performance of MQTT depends highly on the selection of the broker. Mosquitto broker was suitable for constrained devices as it had the lowest hardware requirements but did worse on scenarios with high packet losses. VerneMQ and EMQX were resource hungry but had better latency results.

[16] performed a similar experiment where six MQTT brokers were stress tested. Lightweight brokers such as Mosquitto didn't scale well but performed better on constrained devices, whereas scalable brokers were able to perform better on improved multi-core hardware.

[3] experimentally benchmarked five open-source MQTT Brokers in an edge computing scenario with constrained devices. Payload sizes and corresponding performance offset response time were evaluated. They found that network parameters and the hardware have an impact on performance of the broker.

[8] compared two most used MQTT brokers, Mosquitto and RabbitMQ, in the scenario of smart cities. They found that the hardware used to deploy the broker has influence over the message publishing performance of the broker. Also, Mosquitto achieved better results in scenarios where there was not a lot of data flow.

[11] measured the power consumption of MQTT protocol with and without using security mechanisms such as TLS. They found using TLS protocol reduced the performance and resulted in higher energy consumption in constrained devices. However, the study focused on one broker instead of performing comparative analysis of multiple brokers.

Although the performance of MQTT brokers has been widely studied, there is still a lack of research concerning energy consumption of MQTT brokers. This research aims to introduce a factor of energy efficiency when performing comparative analysis of MQTT brokers.

D. Benchmarking tools

Benchmarking tools allow standardized testing of a MQTT system with various configurations and scenarios. [15] used Apache JMeter to handle dynamic MQTT scenarios. An open-source load testing tool MQTTLader has been used to evaluate performance of MQTT brokers with functionality such as shared subscription [17].

eMQTT-Bench is another protocol benchmarking tool maintained by EMQ Technologies and written in Erlang programming language. It provides various parameter tuning to vary the conditions under which the brokers are tested [6].

E. Measuring energy consumption

Hardware based power measurements tools provide more accuracy than software counterparts [18]. This involves plugging the device into the power-meter and reading the measurements manually. However, the power of individual components

cannot be derived from this as we get the overall power of the device. This drawback has been ignored and a physical power meter has been used in this research, due to the focus on overall energy consumption instead of specific components.

III. METHODOLOGY

A. Macro methodology

The methodology of this research is organized into several steps:

- 1) The research started with defining clear research goals or questions to focus the research to a specific problem.
- 2) A literature review of various MQTT brokers was conducted concerning performance and energy efficiency of these brokers. Also, review of various benchmarking tools and power measuring tools was done at this step.
- 3) Following the literature review, an experiment was designed to fulfill the research goals and questions. The tools, hardware and the tests were selected based on the information gathered from the literature review. The experiment was conducted multiple times to ensure that the test was not influenced by external factors and to minimize room for error.
- 4) After the execution of the tests, data was collected and recorded. Relevant performance metrics as well as power measurements for each trial were taken and compiled into a CSV file. This information was used to perform statistical analysis and description using methods such as ANOVA. The findings were interpreted to draw conclusions and provide answers for the formulated research questions. Also, recommendations were provided based on the conclusions.

B. Infrastructure Architecture

This section describes the architecture used to perform the experiments. The brokers were hosted on a Raspberry-pi 4 Model B device using a containerization tool Docker. To measure the power consumption, a power meter was connected to the device before plugging it to the socket. A separate environment was used to host the benchmarking tool, to minimize its effect on the power consumption. The two environments used Wireless LAN to publish and subscribe to MQTT requests. Figure 1 demonstrates the overall architecture of the experiment.

Specifications of the system are provided in table II.

TABLE II: Specifications of Raspberry Pi 4 and Laptop

Raspberry Pi 4	
Hardware	Raspberry Pi 4 Model B
OS	Raspbian OS 64-bit
CPU Architecture	ARMv8 (64-bit)
RAM	8 GB
Storage	16 GB
Laptop	
Hardware	MacBook Air M1
OS	Sonoma 14.2.1
CPU Architecture	Apple Silicon M1 (ARMv8.5-A)
RAM	8 GB
Storage	256 GB

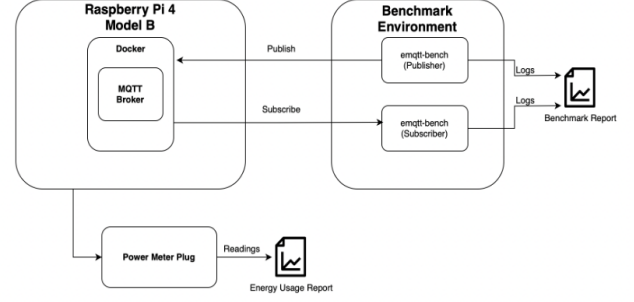


Fig. 1: Architecture diagram of experiment.

C. Micro Methodology

a) *Experimental Design*: For this experiment, 4 different brokers were chosen based on their language of implementation and scalability. Two different benchmark processes were run on the laptop to publish and subscribe simultaneously. The publishing process generates messages based on the parameters. In the testing scenario, 20 clients generate a 100-byte message every 100ms. This results in 200 messages sent per second. The published messages are sent to the broker on the raspberry pi using local wireless network.

The raspberry pi has a single broker running using a docker container. The broker receives the messages sent by the publishing process. The messages are then sent back to the subscribing process on the laptop. There are 500 clients that subscribe to these messages using the topic. After receiving the messages, the benchmark process logs the performance metrics.

The whole process was run for 20 seconds at a time for each broker. The corresponding power and performance metrics were taken and tabulated into a CSV file. The experiment was repeated 10 times. After each trial the docker container is shut off and the raspberry pi is allowed to come back to idle power consumption of 2.0 watts to ensure that the readings are consistent. The configuration used for publishing and subscribing are given in Table 3.3.

TABLE III: Publishing parameters for eMQTT-bench

Parameter	Value
Topics	1
MQTT Version	5
Message Size	100 bytes
QoS (Quality of Service)	0
Publishing Clients	20
Message Rate	1/100ms
Retain Messages	FALSE
TLS/SSL	FALSE

TABLE IV: Subscribing parameters for eMQTT-bench

Parameter	Value
Topics	1
MQTT Version	5
Receiving Clients	500

b) *Analyzing and Comparing*: The obtained data was used to visualize the performance and energy usage trends

for each broker. Further, statistical analysis was performed to determine whether there was significant difference in the energy consumption, publishing performance and subscribing performance between the four brokers. The results of the analysis are discussed in the next section.

IV. FINDINGS AND DISCUSSION

A. Performance

Table V and VI provide the performance results for each broker. The tables provide the total published and received messages by each broker and the rate of publishing and receiving. The published or received messages at any instant is dependent on the current rate.

It can be observed from the table that the rate of publishing the messages thereby the total published messages is similar across all the brokers. Each broker published around 4000 messages at the end of each iteration. However, there is significant variation between the brokers when receiving the messages. It can be observed that NanoMQ has a higher performance when receiving messages compared to the other brokers. Also, even though the receiving rate for the three other brokers were the same, Mochi-MQTT had received significantly less messages on average.

TABLE V: Total published and received messages for each broker.

Broker	Total Published (Messages)	Total Received (Messages)
EMQX	3964.6	371094.9
HiveMQ CE	3959.6	375782.8
Mochi-MQTT	3983.7	216664.6
NanoMQ	3957.4	454560.5

TABLE VI: Publish and receive rate for each broker.

Broker	Publish Rate (Messages/s)	Receive Rate (Messages/s)
EMQX	198.19	18553.25
HiveMQ CE	197.94	18788.12
Mochi-MQTT	199.14	18627.77
NanoMQ	197.83	22646.69

ANOVA test was carried out to see whether the publishing and receiving performance of the brokers was statistically different. We consider the significance level, $\alpha=0.05$. First, the Total published messages were compared with the formulation of the following hypothesis:

- **Null Hypothesis:** The total published messages means are equal for each broker.
- **Alternative Hypothesis:** The total published messages vary significantly between the brokers.

From the test we obtain P-value = 0.431439. In this case, the null hypothesis is accepted because the P-value (0.431439) is greater than the significance level. This suggests that there is no significant difference between the publishing performance of the brokers.

Secondly, the total received messages was compared with the formulation of the following hypothesis:

- **Null Hypothesis:** The total received messages means are equal for each broker.

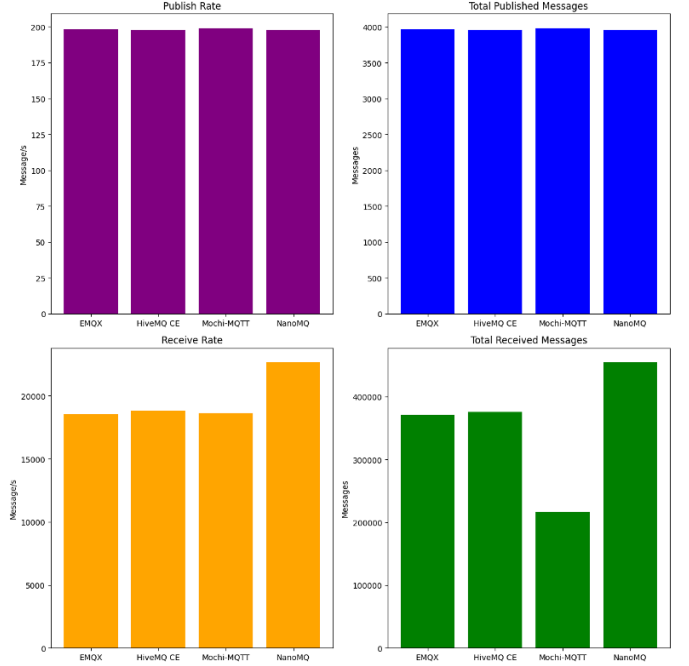


Fig. 2: Performance comparison for all brokers

- **Alternative Hypothesis:** The total received messages vary significantly between the brokers.

From the test we obtain P-value = 0.0015. In this case, the null hypothesis can be rejected because the P-value (0.0015) is below the significance level. This indicates significant differences in the subscribing performance of at least one of the brokers.

B. Energy Consumption

Table VII and Figure 3. look into the power and energy consumption of the brokers. The readings were initially taken in watts and later converted to Joules/s and Watt/h using computational tools. The idle average wattage for the raspberry pi was observed to be 2.04 Watts without running the container for the broker or any other background process. The watt readings were taken after running the container for each broker individually and using the benchmarking tools to stress test the broker. The average of the readings is taken across all trials and tabulated in Table 4.3.

It can be observed that broker HiveMQ CE consumes the most energy out of all the brokers followed by EMQX. Mochi-MQTT and NanoMQ both consume significantly less energy with NanoMQ consuming the least. This can be attributed to the fact that both NanoMQ and Mochi-MQTT are meant to be lightweight and embeddable brokers whereas EMQX and HiveMQ CE are high performance scalable brokers.

Similarly, the carbon footprint for the brokers has been calculated by converting Wh metric to CO₂ gram. The conversion factor was obtained from the Greenhouse conversion factors report provided by the U.K. government. The value was observed to be 0.2070 kg CO₂e (Department for Energy Security and Net Zero, 2023). As is observed with the en-

ergy consumption, NanoMQ had the lowest carbon footprint, whereas HiveMQ CE had the highest carbon footprint.

TABLE VII: Power consumption of all brokers.

Broker	Watts	Energy (J/s)	Energy (Wh)	CO2 Emissions (gm)
EMQX	4.707	94.14	0.02615	0.00541305
HiveMQ CE	5.099	101.98	0.028328	0.00586385
Mochi-MQTT	4.0625	81.25	0.022569	0.004671875
NanoMQ	4.003	80.06	0.022239	0.00460345

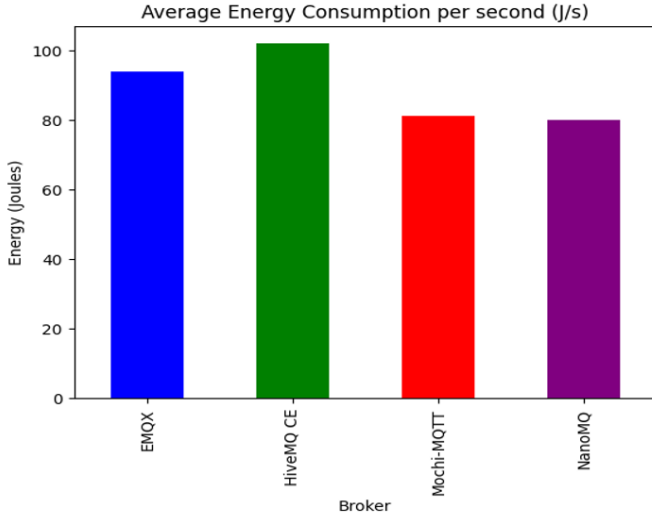


Fig. 3: Comparison of Average Energy Consumption (Joules/s)

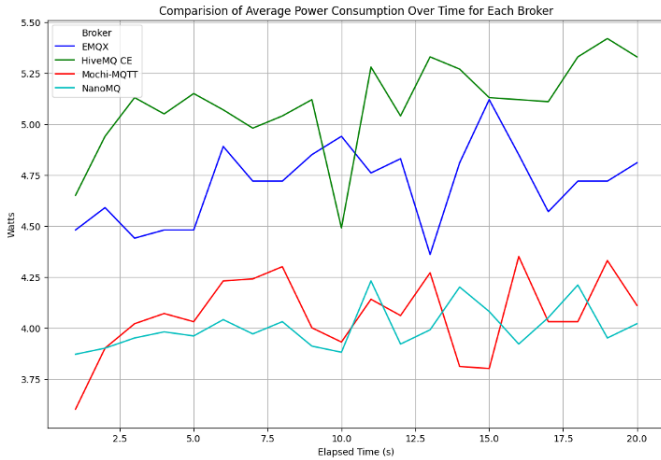


Fig. 4: Comparison of average power consumption over time for each broker

An ANOVA test was performed to see if the energy consumption means of the brokers was different. The following hypothesis was formulated for the case:

- **Null Hypothesis:** The average energy consumption for each broker is equal.
- **Alternative Hypothesis:** The average energy consumption for each broker differs significantly.

From the test we obtain P-value = 6.7161E-120. The P-value obtained is lower than the significance value of 0.05.

This suggests that there are significant differences between the average amount of energy consumed by each broker during operation.

Figure 4 shows the energy consumed by the brokers over time. It can be observed that NanoMQ and Mochi-MQTT mostly fall under 4.25 watts with some spikes to 4.30 watts. Mochi-MQTT has higher fluctuations but requires less initial wattage. On the other hand, HiveMQ CE can be seen to have the spikes to over 5.40 watts but on average staying just under 5.25 watts. EMQX showed slightly better efficiency with a just few spikes to over 5.00 watts.

V. CONCLUSION AND RECOMMENDATIONS

With the widespread growth of Internet of Things (IoT), there has been a necessity to study the energy efficiency of various IoT components. In this study, the performance and energy efficiency of four different prominent MQTT brokers was evaluated in the context of constrained hardware.

From the results, it was observed that the publishing performance of all brokers were similar. However, there was a significant difference in the receiving performance, as NanoMQ consistently performed better than other brokers. The lightweight brokers performed on par with the high-performance brokers on both publishing and subscribing tasks. This could be attributed to the raspberry pi not having enough computational power to allow full features of the high-performance brokers. This is in-line with previous research that had been performed under similar conditions. In terms of energy consumption, the lightweight brokers, NanoMQ and Mochi-MQTT, were very efficient and had the least amount of CO2 emissions. HiveMQ CE was on the opposite end of the spectrum with the highest carbon footprint, followed by EMQX.

A. Recommendation

In terms of deploying an MQTT broker to a resource constrained device, it is better to choose a broker that is optimized for the hardware. A constrained device may not be able to provide enough computational power to high-performance brokers. Also, it is necessary to choose a broker that has better energy efficiency. Hence, out of the evaluated brokers, NanoMQ is recommended as it had the lowest energy consumption while maintaining similar performance to other brokers. It is thus recommended to introduce energy consumption as an evaluating criterion as it can provide insights into how different brokers perform under different hardware resources.

B. Future Work

Further studies can be performed comparing the energy efficiency of the brokers considering the QoS parameters, security mechanism and various other MQTT parameters. Also, the benchmarking scenario can be varied to test the broker under different conditions; realistic conditions where the data rate is lower but there are high numbers of devices publishing and subscribing or even more stressful conditions where both the clients and subscriber number are increased

to millions. More brokers can be evaluated for efficiency than what was selected for this study. Research also can be performed to study if the programming language influences the energy efficiency of the brokers. The research into efficiency analysis of IoT infrastructure has a lot of potential as not much research has been conducted to date regarding this.

REFERENCES

- [1] A. Lukyanov, D. Donskoy, M. Vernezi, and D. Karev, "Estimation of the carbon footprint of iot devices based on esp8266 microcontrollers," vol. 279. EDP Sciences, 7 2021.
- [2] R. Canek, P. Borges, and C. Taconet, "Analysis of the impact of interaction patterns and iot protocols on energy consumption of iot consumer applications," pp. 1–17, 2022. [Online]. Available: <https://hal.science/hal-03710735>
- [3] J. Dizdarevic, M. Michalke, and A. Jukan, "Engineering and experimentally benchmarking open source mqtt broker implementations," 5 2023. [Online]. Available: <http://arxiv.org/abs/2305.13893>
- [4] D. Trihinas, L. Thamsen, J. Beilharz, and M. Symeonides, "Towards energy consumption and carbon footprint testing for ai-driven iot services." Institute of Electrical and Electronics Engineers Inc., 2022, pp. 29–35.
- [5] E. Bertrand-Martinez, P. D. Feio, V. de Brito Nascimento, F. Kon, and A. Abelém, "Classification and evaluation of iot brokers: A methodology," vol. 31. John Wiley and Sons Ltd, 5 2021.
- [6] E. Team, "Performance test with emqtt-bench," 2023. [Online]. Available: <https://www.emqx.io/docs/en/latest/performance/benchmark-emqtt-bench.html>
- [7] M. O. A. Enany, H. M. Harb, and G. Attiya, "A comparative analysis of mqtt and iot application protocols." Institute of Electrical and Electronics Engineers Inc., 7 2021.
- [8] D. L. D. Oliveira, A. F. Da, S. Veloso, J. V. V. Sobral, R. A. L. Rabêlo, J. J. P. C. Rodrigues, and P. Solic, "Performance evaluation of mqtt brokers in the internet of things for smart cities," 2019.
- [9] H.-Y. Chien, G.-H. Qiu, R.-W. Hung, A.-T. Shih, and C. Su, "Hierarchical mqtt with edge computation," 2019.
- [10] A. S. S. Ansyah, M. Arifin, M. B. Alfian, M. V. Suriawan, N. H. Farhan-syah, A. M. Shiddiqi, and H. Studiawan, "Mqtt broker performance comparison between aws, microsoft azure and google cloud platform." Institute of Electrical and Electronics Engineers Inc., 2023.
- [11] A. R. Alkhafajee, A. M. A. Al-Muqarm, A. H. Alwan, and Z. R. Mohammed, "Security and performance analysis of mqtt protocol with tls in iot networks." Institute of Electrical and Electronics Engineers Inc., 2021, pp. 206–211.
- [12] "Mochi-mqtt server." [Online]. Available: <https://github.com/mochi-mqtt/server>
- [13] E. Technology, "Introduction." [Online]. Available: <https://nanomq.io/docs/en/latest/>
- [14] E. Technologies, "Emqx overview." [Online]. Available: <https://www.emqx.io/docs/en/latest/>
- [15] M. Bender, E. Kirdan, M. O. Pahl, and G. Carle, "Open-source mqtt evaluation." Institute of Electrical and Electronics Engineers Inc., 1 2021.
- [16] B. Mishra, B. Mishra, and A. Kertesz, "Stress-testing mqtt brokers: A comparative analysis of performance measurements," *Energies*, vol. 14, 9 2021.
- [17] R. Banno, K. Ohsawa, Y. Kitagawa, T. Takada, and T. Yoshizawa, "Measuring performance of mqtt v5.0 brokers with mqttloader." Institute of Electrical and Electronics Engineers Inc., 1 2021.
- [18] G. Bekaroo and A. Santokhee, *Power Consumption of the Raspberry Pi: A Comparative Analysis*, 2016.